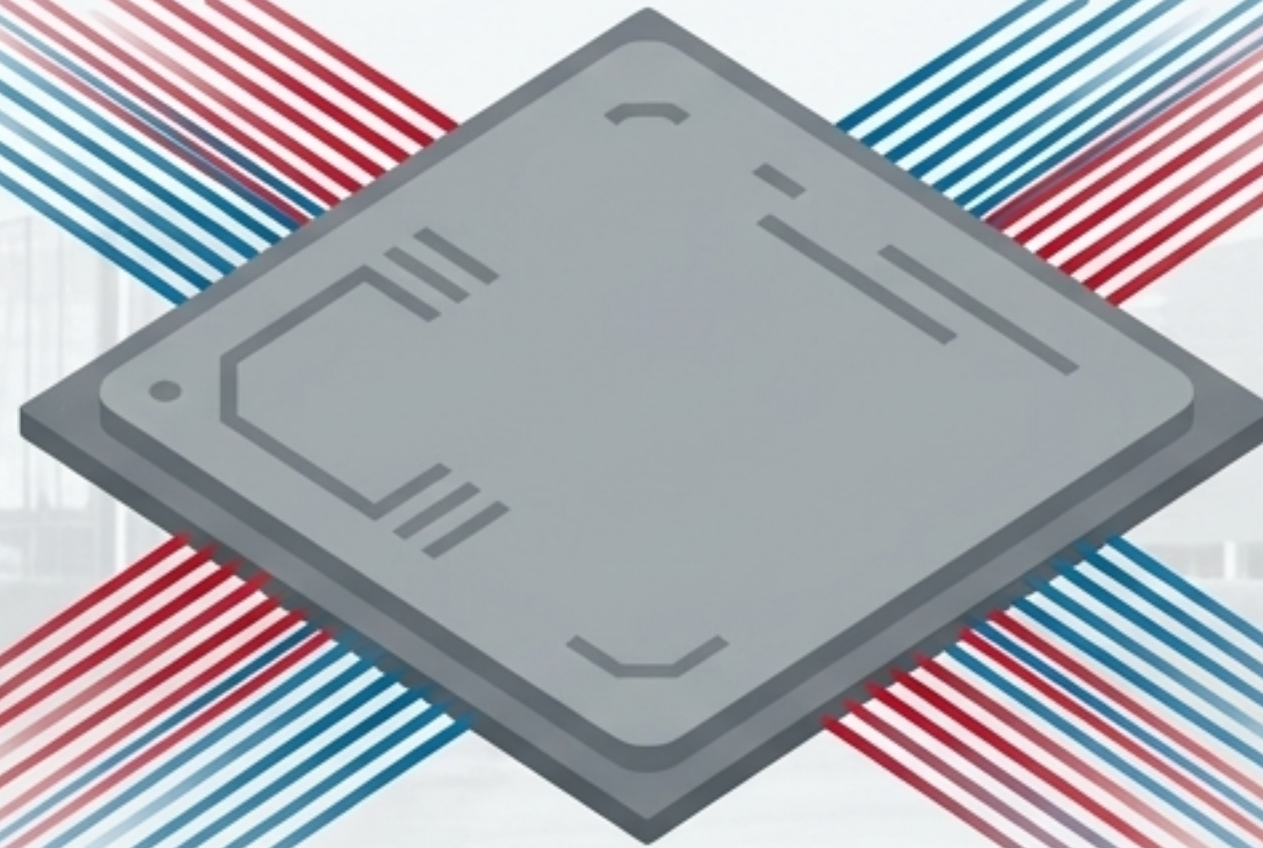


# De Snelheidsrace: Een Reis door Moderne Processorarchitectuur

Van de basisprincipes tot de grenzen van Artificiële Intelligentie.



Een overzicht van de kernconcepten uit computerarchitectuur, ontworpen als een zelfstandige educatieve gids. We volgen het verhaal van hoe we processoren steeds sneller hebben gemaakt, de uitdagingen die we tegenkwamen en de ingenieuze oplossingen die de moderne computerwereld hebben gevormd.



# De Anatomie van een Processor: De Drie Pilaren

Elke processor, van de kleinste microcontroller tot de krachtigste server-CPU, is opgebouwd uit drie logische eenheden die samenwerken om instructies uit te voeren.

- **Besturingseenheid (Control Unit)**

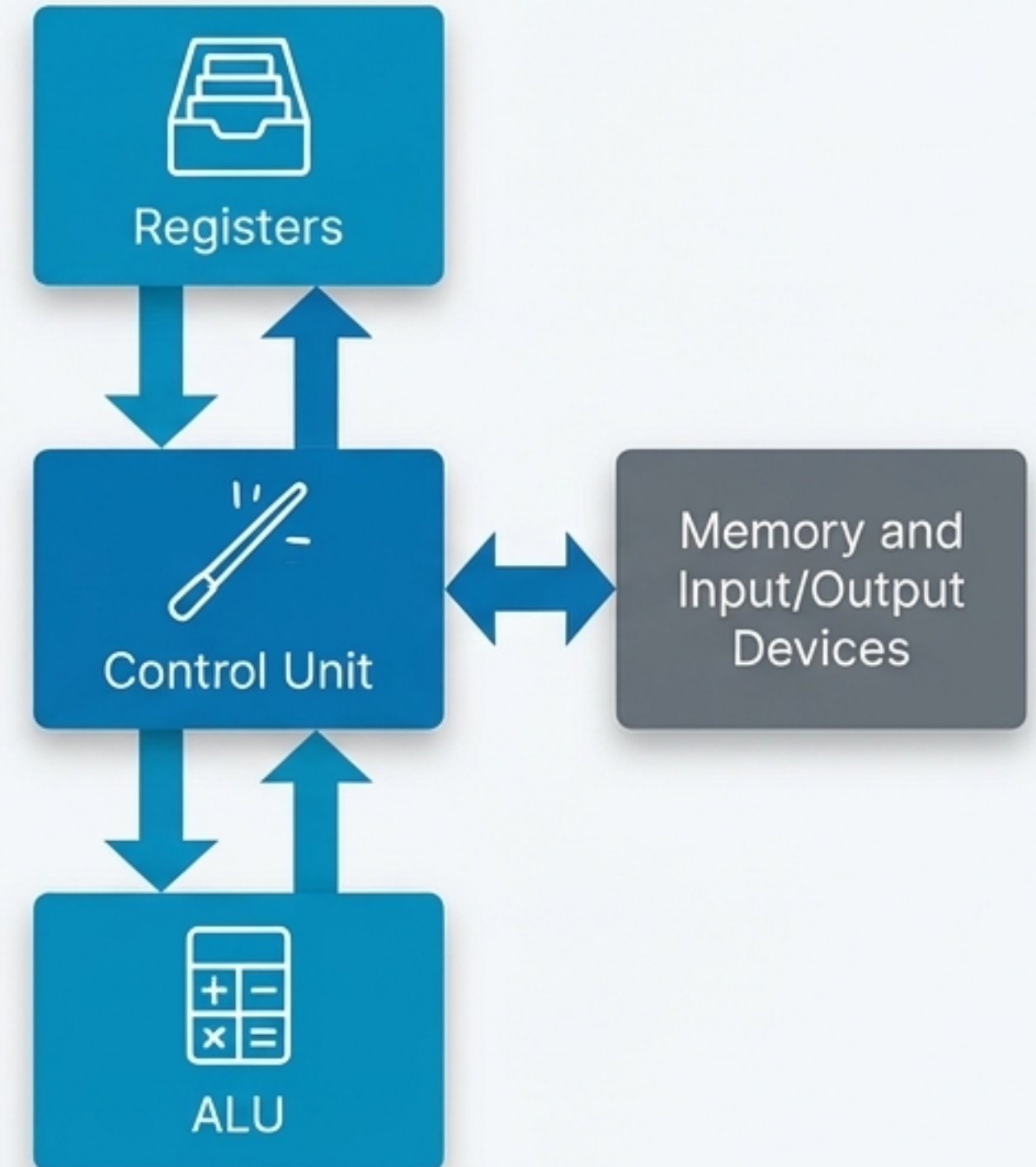
De 'dirigent'. Haalt en decodeert instructies en stuurt de operaties van de hele processor aan.

- **Rekenkundig-logische Eenheid (ALU)**

Het 'rekenwonder'. Voert alle wiskundige (optellen, aftrekken) en logische (AND, OR, NOT) bewerkingen uit.

- **Registers**

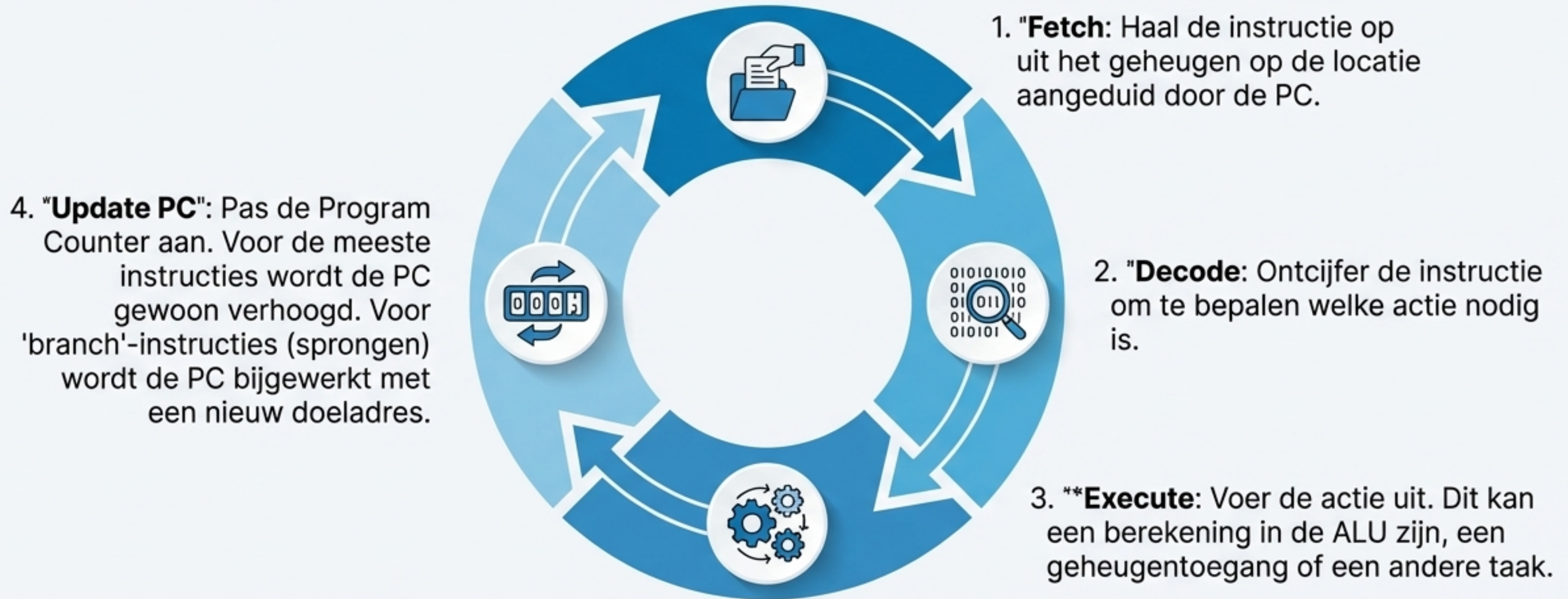
Het 'kortetermijngeheugen'. Ultrasnelle, tijdelijke opslaglocaties direct in de processor voor data en resultaten.





# De Hartslag van de Machine: De Instructiecyclus

De Besturingseenheid volgt een repetitieve cyclus om software-instructies om te zetten in actie. Deze cyclus is de basis van alles wat een computer doet. De Program Counter (PC) register bevat altijd het adres van de *volgende* uit te voeren instructie.



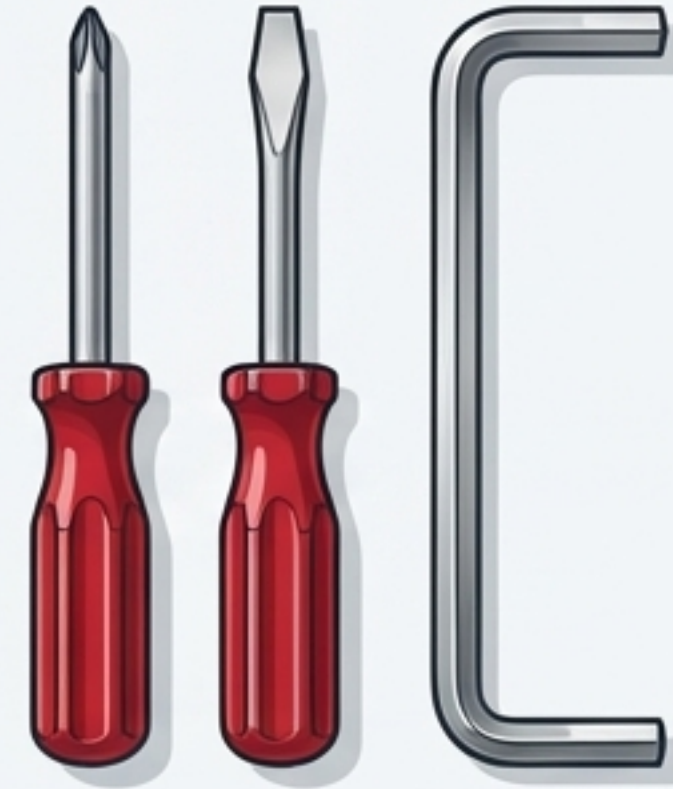


# Een Fundamentele Keuze: Complexiteit vs. Eenvoud (CISC vs. RISC)



## CISC (Complex Instruction Set Computer)

- **Filosofie:** **Rijke instructieset**. Eén instructie kan meerdere sub-taken uitvoeren (bv. data laden, bewerken, opslaan).
- **Kenmerken:** **Minder registers**, instructies duren meerdere klokcycli.
- **Voorbeeld:** **x86** (de architectuur in de meeste pc's en laptops).



## RISC (Reduced Instruction Set Computer)

- **Filosofie:** **Kleine, eenvoudige instructieset**. Elke instructie doet één simpele taak en doet dat razendsnel.
- **Kenmerken:** **Veel registers**, waardoor geheugentoegang minder vaak nodig is. Geoptimaliseerd voor snelle, afzonderlijke instructies.
- **Voorbeelden:** **ARM, RISC-V** (dominant in mobiele apparaten en embedded systems).



# De Grote Uitdaging: De 'Memory Wall'

De kloksnelheden van processoren zijn de afgelopen decennia exponentieel toegenomen, bijna 1000 keer sneller dan de eerste pc's.

De snelheid van DRAM (het hoofdgeheugen) is veel langzamer geëvolueerd.

Dit creëert een gigantische kloof.



**100x**  
LANGZAMER



Toegang tot  
**processor  
register:** ~1 ns



Toegang tot  
**DRAM:**  
~100 ns

Dit betekent dat een moderne processor het grootste deel van zijn tijd zou doorbrengen met **wachten** op data uit het geheugen. Zonder een oplossing is pure processorkracht nutteloos.



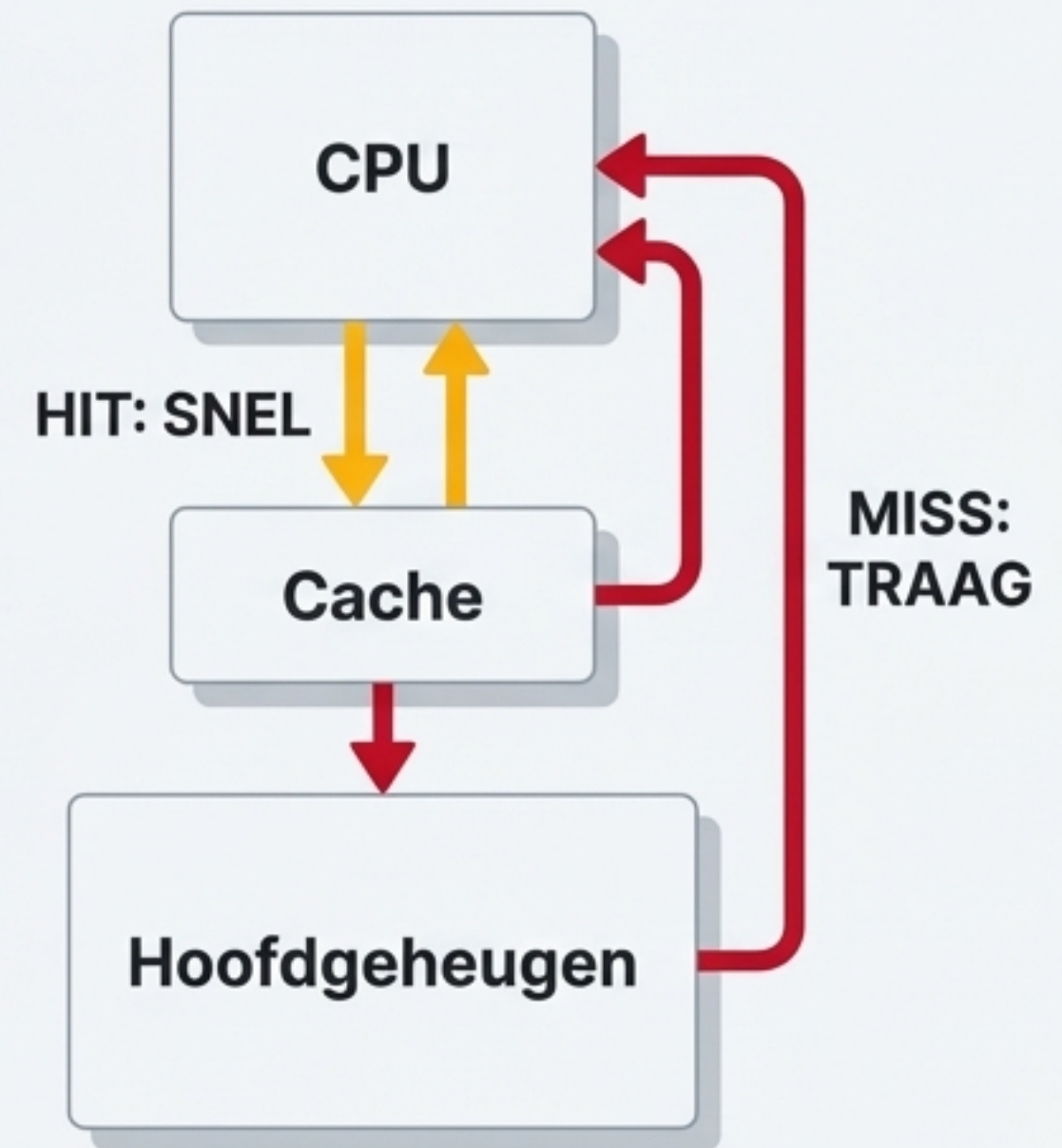
# Oplossing 1: Slimmer Geheugen met Caching

Een cache is een kleine, extreem snelle geheugenbuffer (gemaakt van duur SRAM) die tussen de CPU en het langzame hoofdgeheugen (DRAM) zit. Het doel: de meest gebruikte data en instructies bij de hand houden.

## Het principe: Localiteit van Referentie

- Programma's hebben de neiging om data en instructies te hergebruiken:
  - **Temporele localiteit**: Als een item recent is gebruikt, is de kans groot dat het binnenkort opnieuw wordt gebruikt (bv. een variabele in een lus).
  - **Ruimtelijke localiteit**: Als een item is gebruikt, is de kans groot dat items in de buurt ervan binnenkort worden gebruikt (bv. elementen in een array).

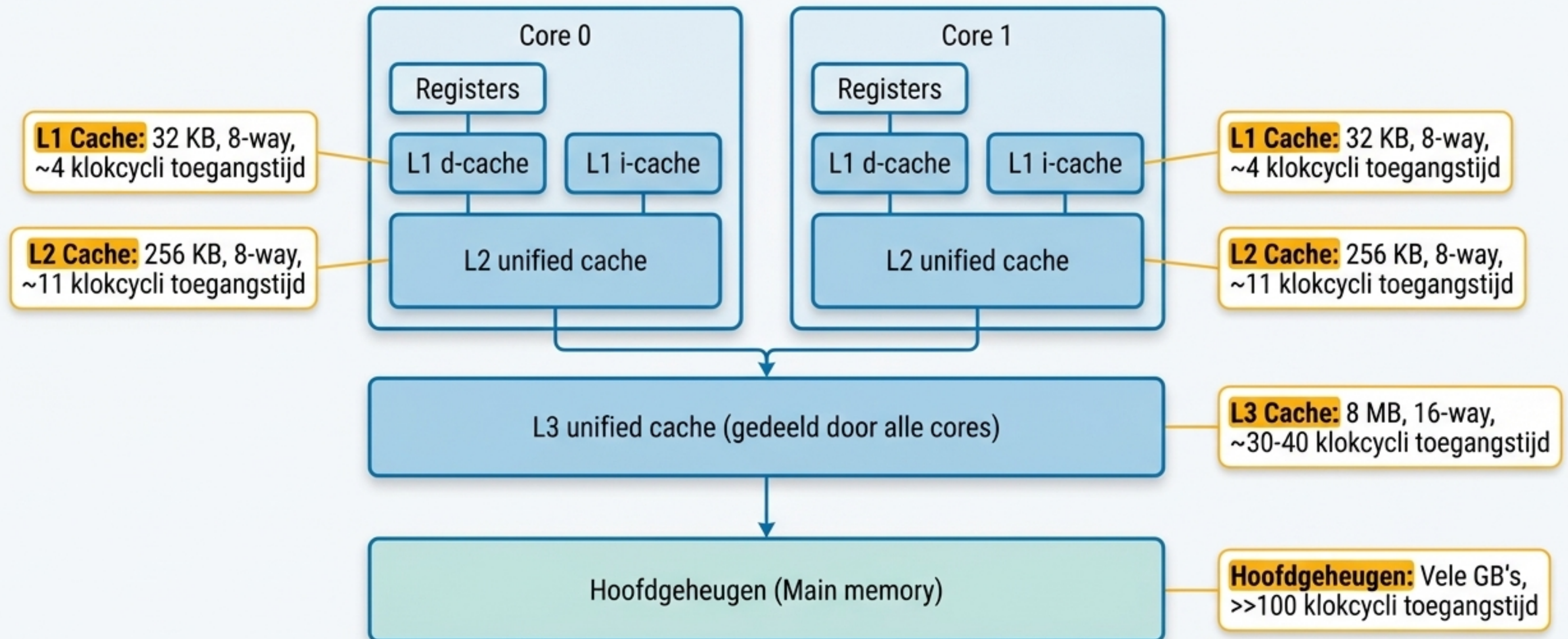
Een **cache hit** (data is in cache) is snel. Een **cache miss** (data moet uit DRAM gehaald worden) is traag, maar de data wordt dan wel in de cache geplaatst voor toekomstig gebruik. Een hit rate van 95-97% is typisch in moderne systemen.





# De Geheugenhiërarchie: Gelaagde Verdediging

Moderne processoren gebruiken meerdere cache-niveaus (levels) om een optimale balans te vinden tussen snelheid en grootte. Elk niveau is groter maar iets trager dan het vorige.



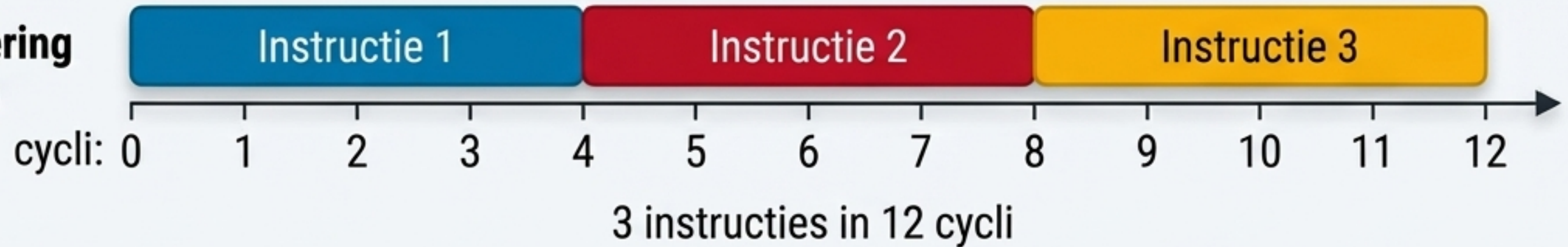


# Oplossing 2: Efficiëntere Uitvoering met Pipelining

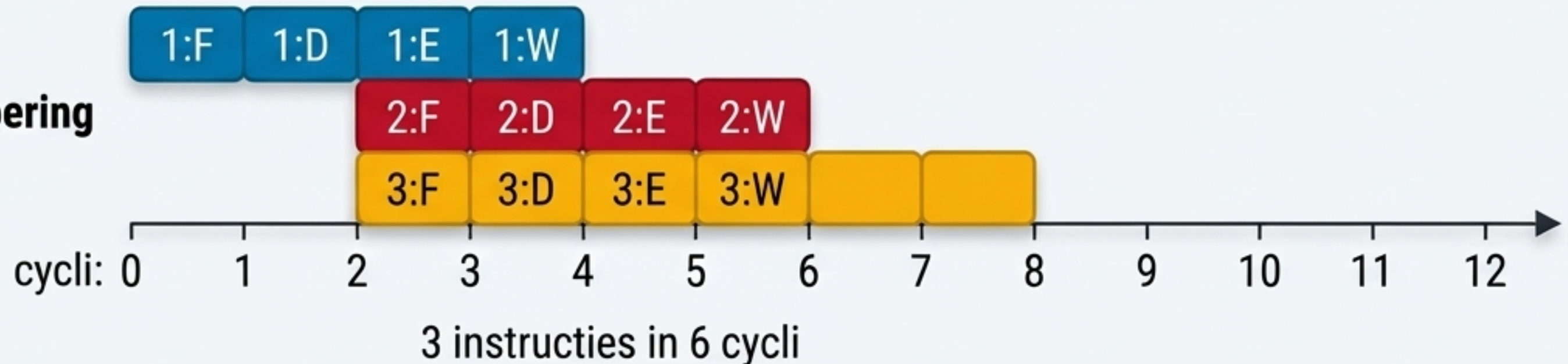
In plaats van een instructie volledig af te handelen voordat de volgende begint, deelt een gepipelineerde processor het werk op in een reeks van stappen, als een lopende band.

De klassieke vier stappen van de pipeline: 1. **Fetch**, 2. **Decode**, 3. **Execute**, 4. **Writeback**

## Sequentiële Uitvoering (Zonder Pipelining)



## Gepipelineerde Uitvoering (Lopende Band)

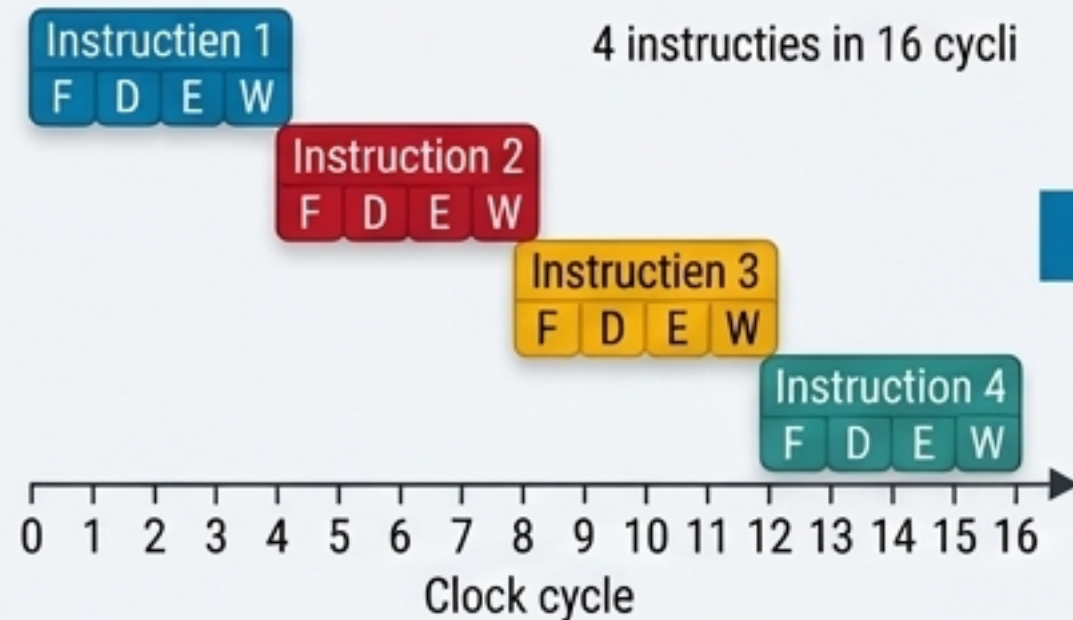




# De Impact van Pipelining: Van 0.25 naar 2.0+ IPC

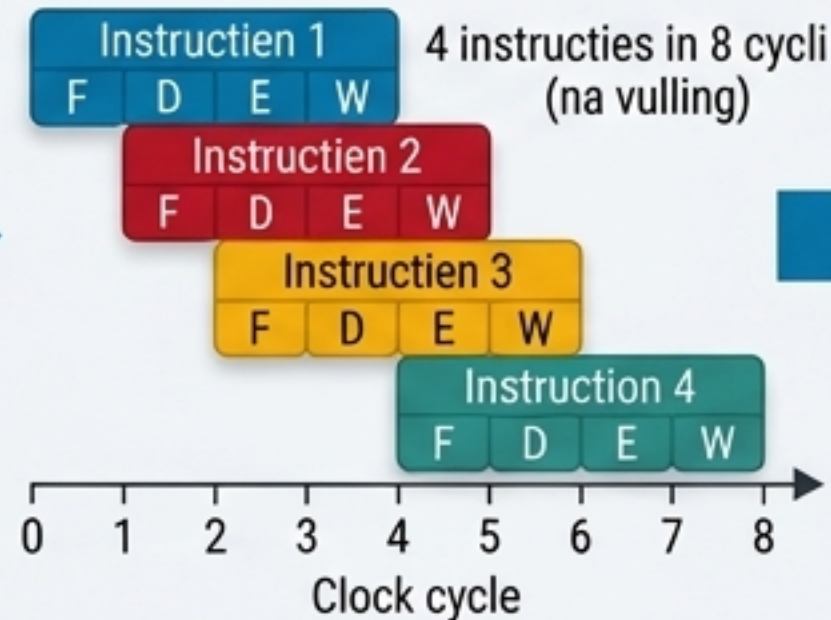
De efficiëntie van een processor wordt vaak gemeten in **IPC (Instructions Per Clock cycle)**. Pipelining zorgt voor een revolutionaire sprong in prestaties.

## Zonder Pipelining



$$\text{IPC} = 0.25$$

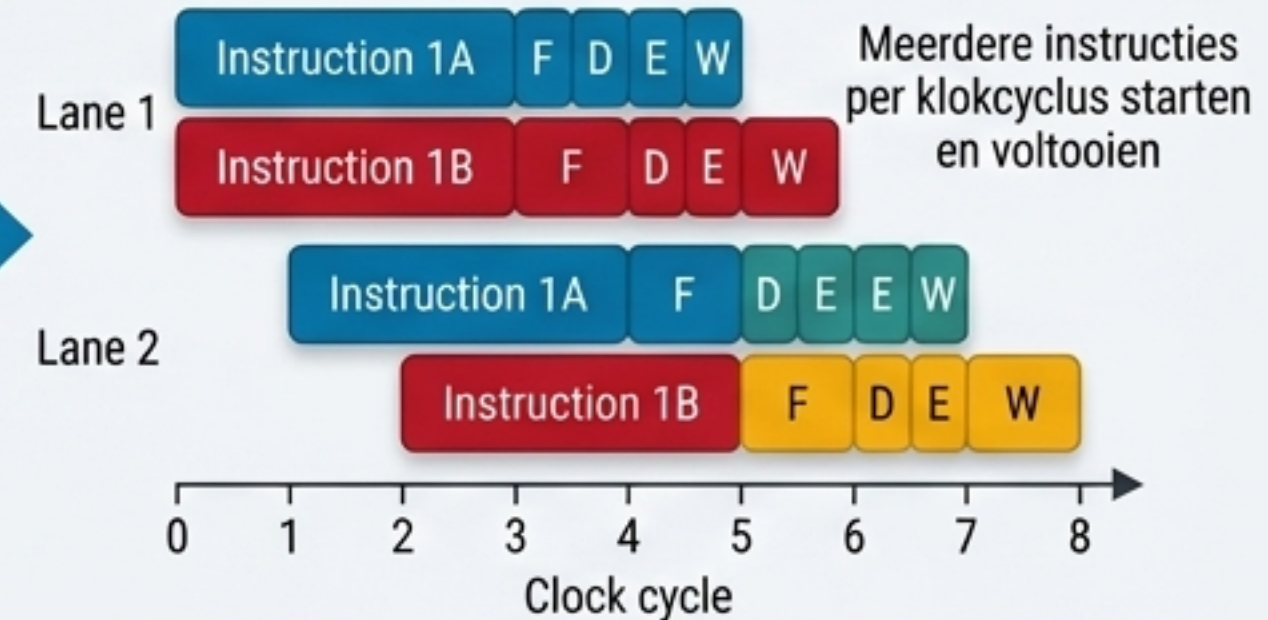
## Met Pipelining



$$\text{IPC} \approx 1.0$$

(een 4x versnelling)

## Superscalar Architectuur



$$\text{IPC} > 1.0$$

### De volgende stap: Superscalar Architectuur

Waarom maar één lopende band? Een superscalar-processor heeft meerdere, parallelle 'execution units' (bv. meerdere ALU's). Dit laat toe om **meer dan één instructie per klokeyclus** te starten.



# De Complexiteit van Snelheid: Pipeline Hazards

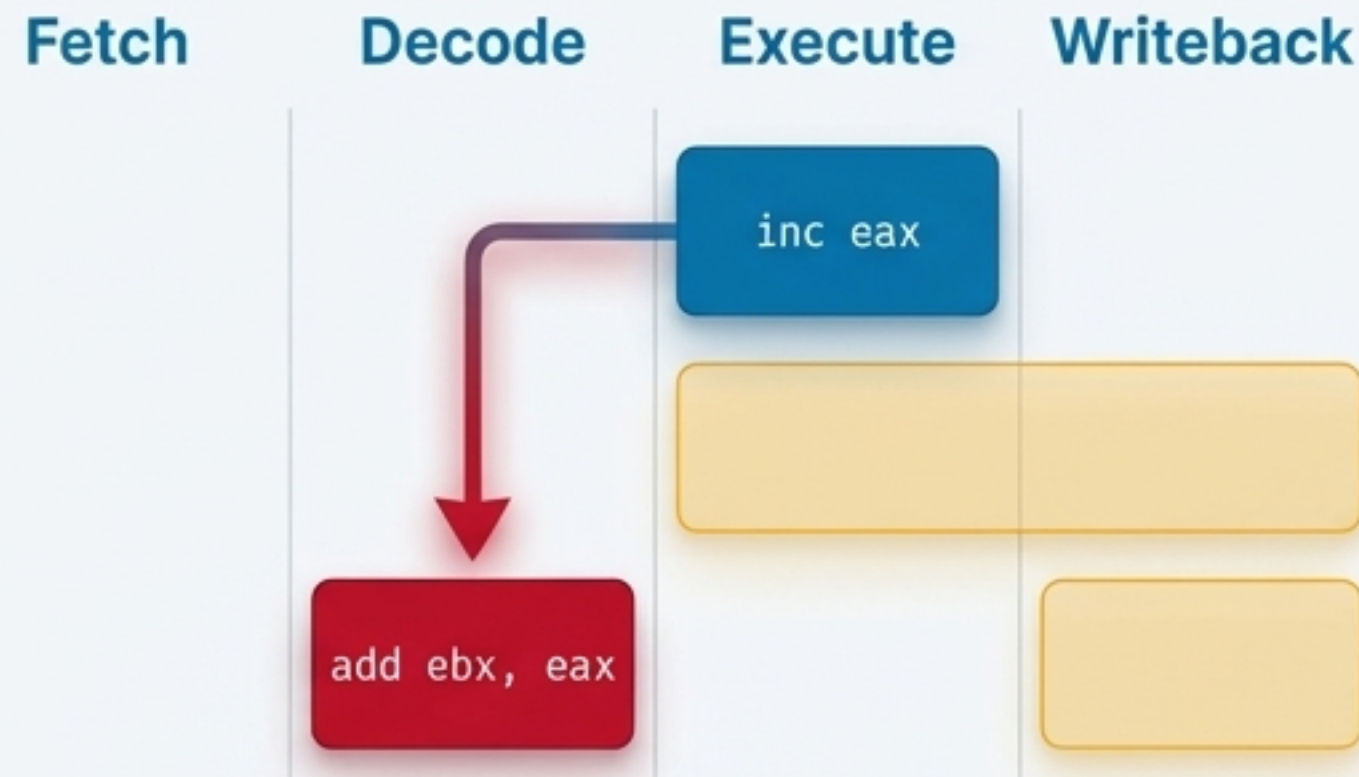
Een diepe en brede pipeline is krachtig, maar loopt tegen problemen aan wanneer instructies van elkaar afhankelijk zijn. Deze '**pipeline hazards**' kunnen de lopende band tot stilstand brengen ('stall' of 'bubble').

## Hazards

- **Data Hazards:** Een instructie heeft het resultaat nodig van een voorgaande instructie die nog niet klaar is.

```
inc eax
add ebx, eax ← "De 'add'
moet wachten op de nieuwe
waarde van 'eax'."
```

- **Control Hazards:** Een conditionele 'branch' (sprong) maakt de volgende instructie onzeker.



## Oplossingen



**Bypassing:** Het resultaat van een ALU-operatie wordt direct doorgestuurd.



**Out-of-Order Execution (OOO):** De processor voert onafhankelijke instructies alvast uit om 'bubbels' te vullen.



**Branch Prediction:** De processor 'gokt' welke kant een branch op zal gaan.



# Voorbij Pure Snelheid: Gegarandeerde Prestaties in Real-Time Systemen

Voor sommige toepassingen is het niet genoeg om *gemiddeld* snel te zijn; elke deadline *moet* gehaald worden. Dit is het domein van real-time computing.

**Hard Real-Time:** Het missen van een deadline is een catastrofale fout.

- **Voorbeelden:** Airbagcontrollers, vliegtuigbesturing, medische apparatuur.

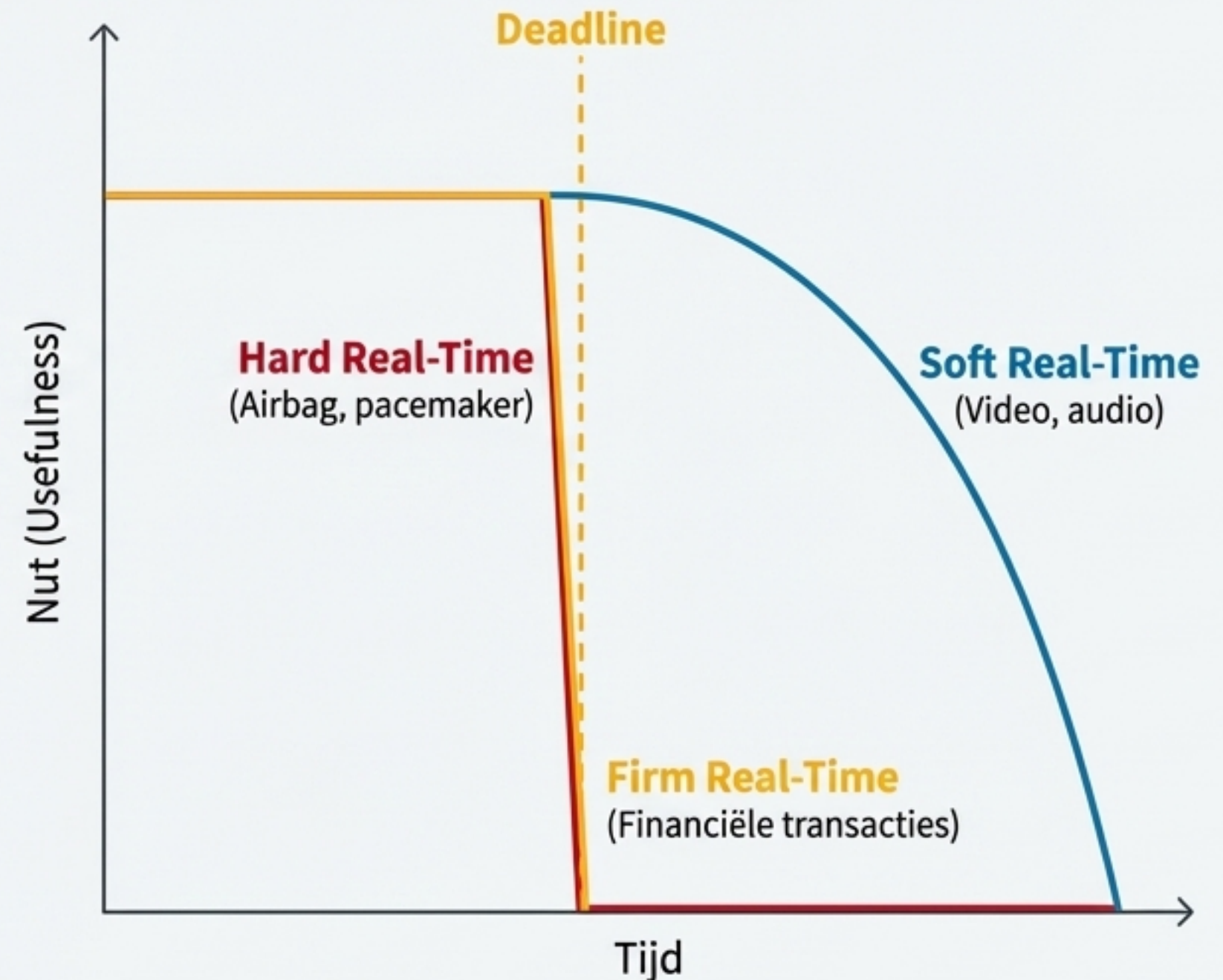
**Soft Real-Time:** Het is acceptabel als een deadline af en toe gemist wordt.

- **Voorbeelden:** Videostreaming, de klok op je telefoon.

**De rol van een RTOS (Real-Time Operating System):**

Een RTOS is een gespecialiseerd besturingssysteem, ontworpen voor determinisme. Het functioneert als een strikte 'verkeersregelaar' met mechanismen zoals:

- **Preemptive Kernel:** Taken met een hogere prioriteit kunnen lagere prioriteitstaken onmiddellijk onderbreken.
- **Mutexes & Semaphores:** Robuuste mechanismen om gedeelde bronnen te beheren.





# De Revolutie van Parallelisatie:

## De GPU (Graphics Processing Unit)

Een GPU is een processor die geoptimaliseerd is om dezelfde operatie gelijktijdig op enorme hoeveelheden data uit te voeren (data-parallelisme). Oorspronkelijk ontworpen voor de miljoenen pixels in een 3D-scène, bestaat de architectuur uit duizenden kleine, gespecialiseerde kernen.



CPU: Ontworpen voor **Lage Latentie**

Enkele, zeer krachtige kernen die complexe, sequentiële taken zo snel mogelijk uitvoeren.



GPU: Ontworpen voor **Hoge Doorvoer (Throughput)**

Duizenden simpele kernen die samenwerken om massieve parallelle taken te verwerken.

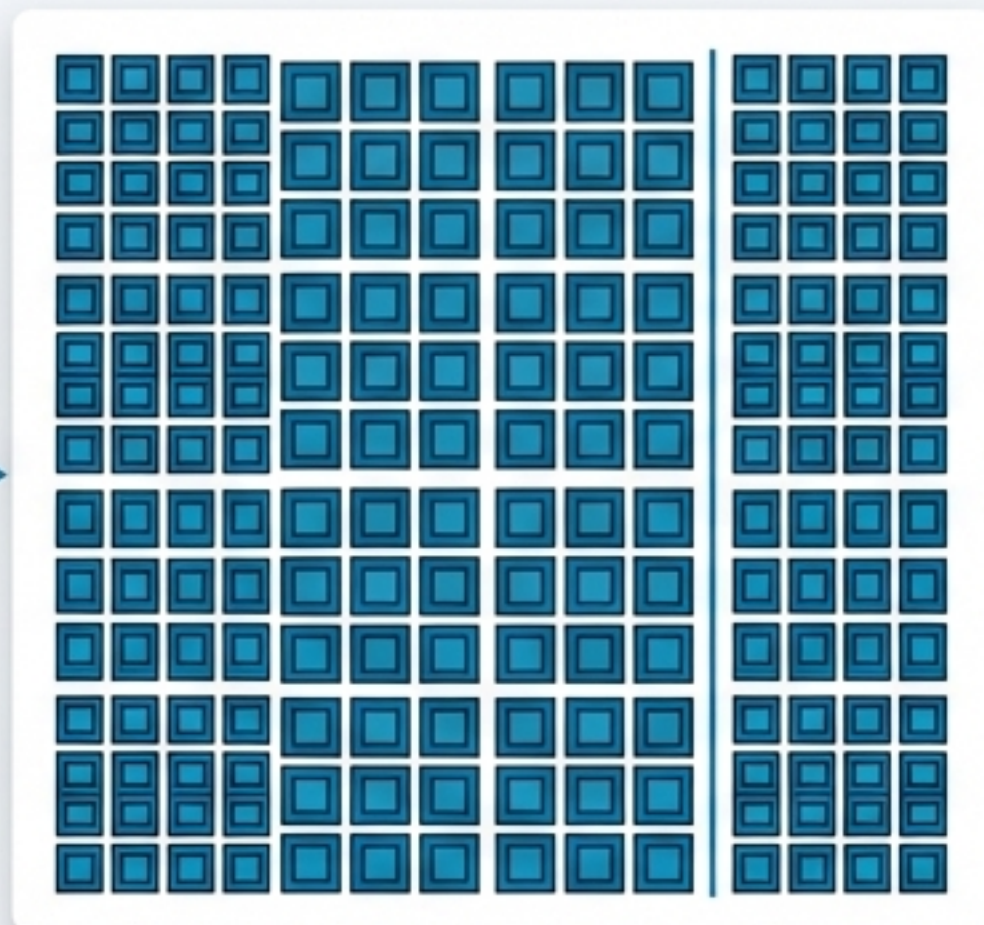


# Van Graphics naar Intelligentie: De GPU als Supercomputer

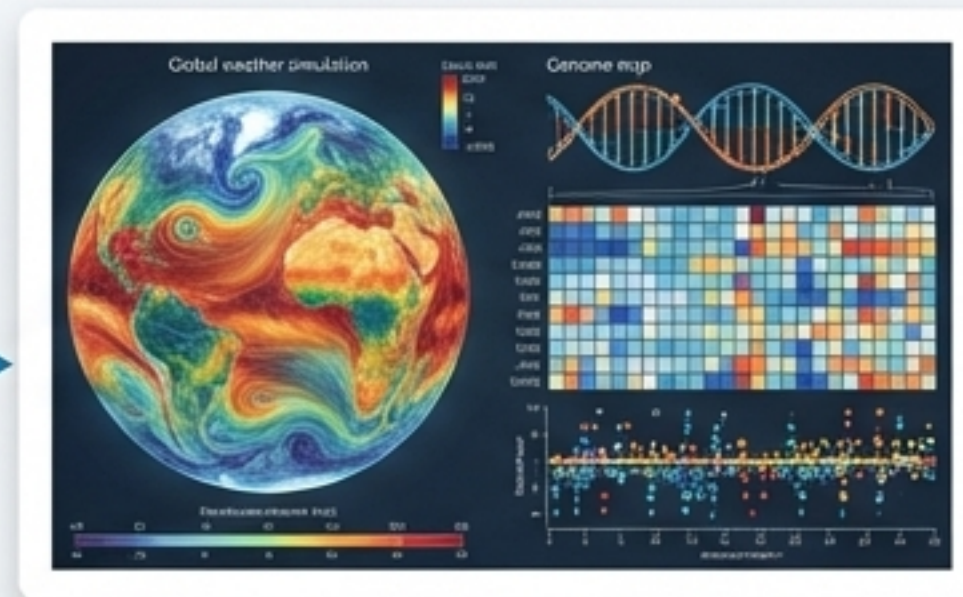
De enorme rekenkracht van moderne GPUs (gemeten in TFLOPS) is herbestemd voor algemene berekeningen (General-Purpose GPU of GPGPU).



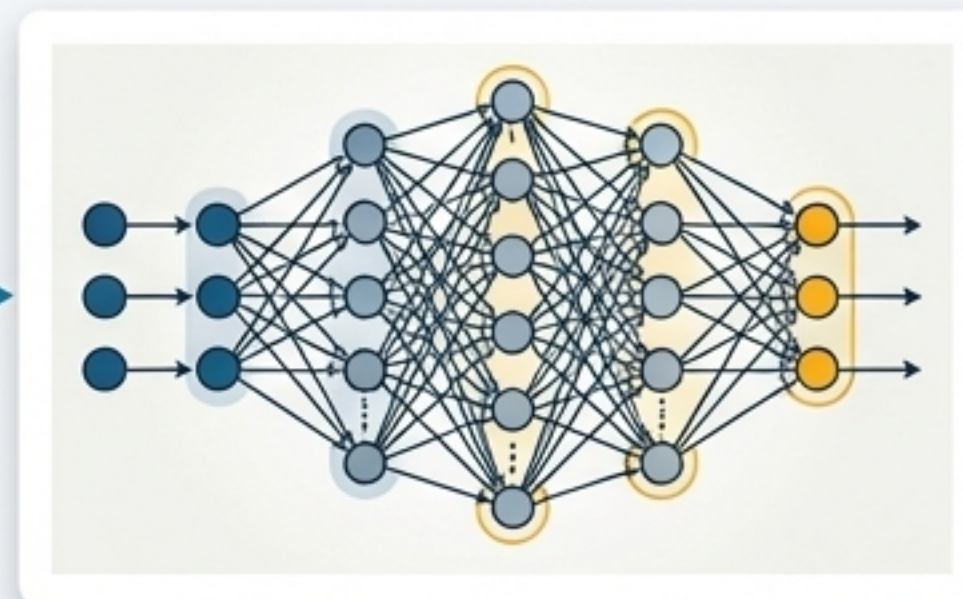
Graphics Rendering



Massieve Parallele Architectuur



Big Data Analyse



Deep Learning (AI)

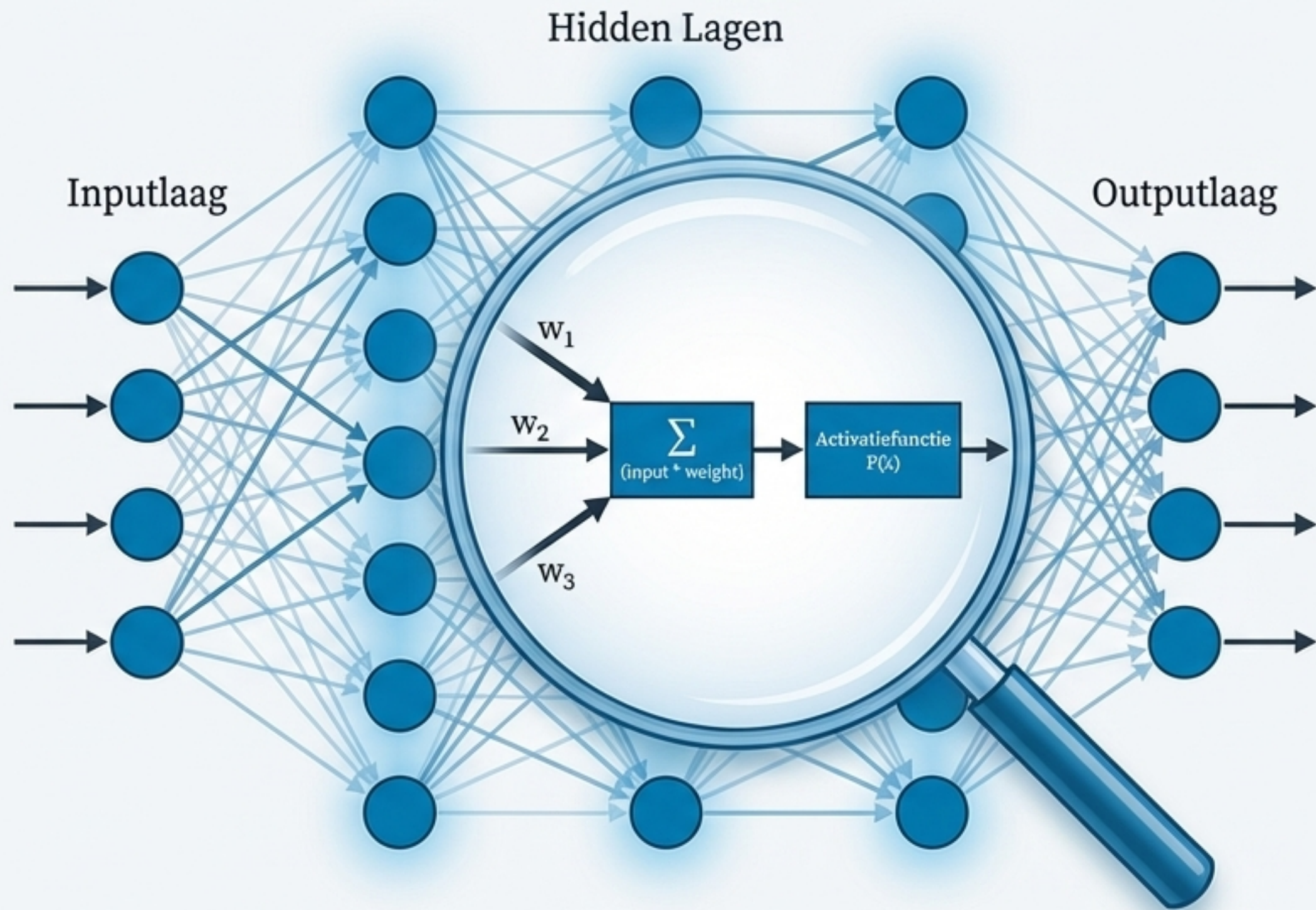
De GPU is een 'kleine supercomputer' geworden die AI-training **honderden keren sneller** kan maken dan een traditionele CPU.



# De Perfecte Match: Neurale Netwerken en Parallelle Architectuur

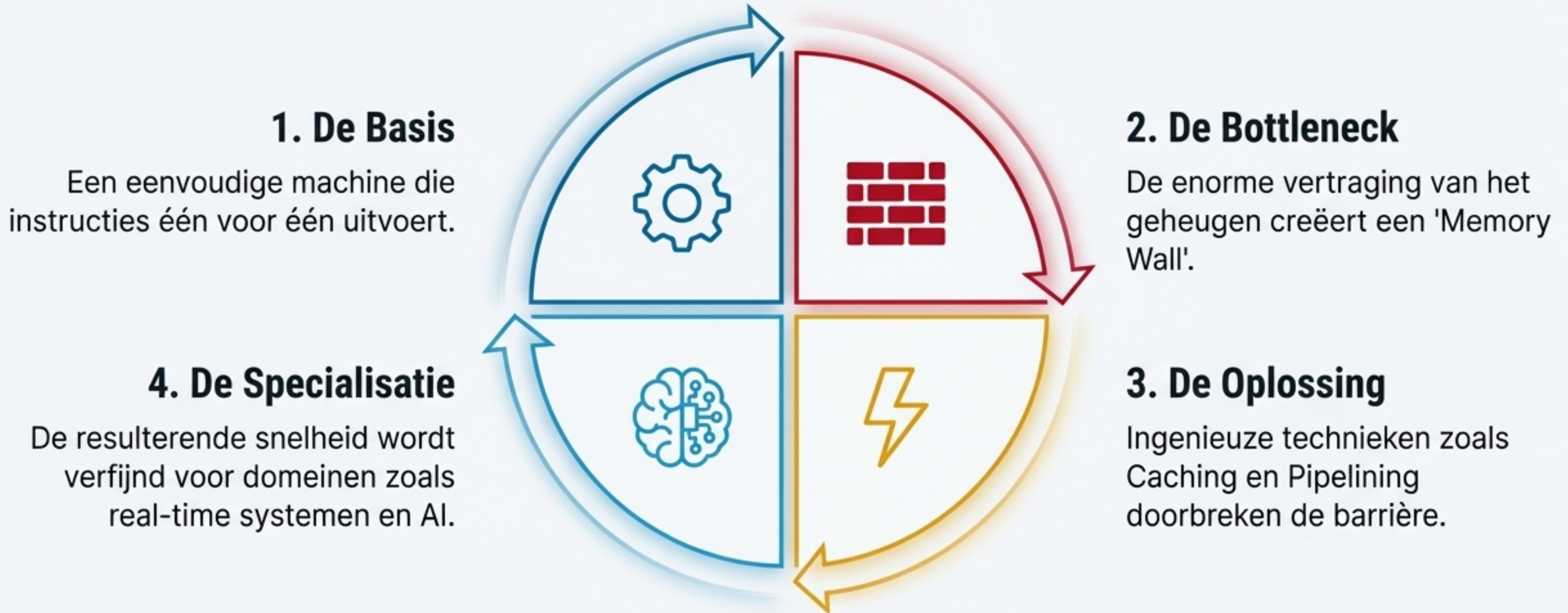
Een Artificieel Neuraal Netwerk (ANN) bootst de werking van het menselijk brein na. Neuronen zijn georganiseerd in lagen. Tijdens de 'forward pass' worden de berekeningen van *alle* neuronen in een laag parallel uitgevoerd.

- **Neuron:** Ontvangt meerdere gewogen inputs, sommeert deze, en past een 'activatiefunctie' toe om een output te genereren.
- **Netwerk:** Neuronen zijn georganiseerd in lagen (input, hidden, output).
- Het trainen van een ANN vereist miljoenen van deze berekeningen. Dit is een taak die enorm versneld wordt door de duizenden kernen van een GPU die deze matrixvermenigvuldigingen tegelijk kunnen uitvoeren.





# De Cirkel van Innovatie: De Voortdurende Snelheidsrace



Onze reis toont een continu patroon: elke oplossing onthult een nieuwe uitdaging. De 'Quest for Speed' is geen lineair pad, maar een eindeloze cyclus van het identificeren van grenzen en het ontwerpen van de volgende generatie slimme architecturen. De vraag is niet *of* er een volgende bottleneck komt, maar *wat* de oplossing zal zijn.